

Left Outer Join in SQL Server with Examples

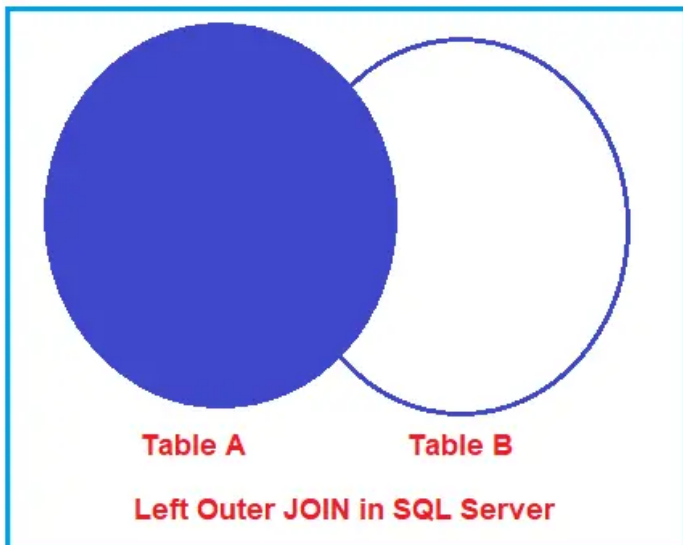
In this article, I will discuss the **Left Outer Join in SQL Server** with Examples. Please read our previous article discussing [SQL Server Inner Join](#) with Examples. You will understand the following pointers in detail at the end of this article.

1. [What is Left Outer Join?](#)
2. [How to implement Left Outer Join in SQL Server?](#)
3. [When do we need to use Left JOIN in SQL Server?](#)
4. [Example to Understand SQL Server Left Outer Join](#)
5. [How can only retrieve non-matching rows from the left table in SQL Server?](#)
6. [Joining three Tables in SQL Server using Left Join](#)

What is Left Outer Join in SQL Server?

A LEFT OUTER JOIN (also known as a LEFT JOIN) in SQL Server returns all records from the left table and the matched records from the right table. If there is no match, the result is NULL on the side of the right table.

That means the Left Outer Join in SQL Server retrieves the matching records from both the tables and the non-matching records from the left side table involved in the JOIN. In that case, the un-matching data will take the null value. The following diagram shows the pictorial representation of the Left Outer Join.



The question that should come to your mind is, which is the left and right table? The answer is the table mentioned to the left of the LEFT OUTER JOIN keyword is the left table, and the table mentioned to the right of the LEFT OUTER JOIN keyword is the right table.

How to implement Left Outer Join in SQL Server?

Please look at the following image, which shows the syntax of Left Outer Join in SQL Server. You can use the **Left Outer Join** or **Left Join** keywords to perform the Left Outer Join Operation in SQL Server. If you use only the **Left JOIN** keyword, it will also perform the Left Outer Join Operation.

Syntax1:

```
SELECT column_name(s)
FROM table1
LEFT OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

Syntax2:

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Here's a breakdown of the components of the SQL statement:

- **SELECT:** Specify the columns you want to retrieve from both tables.
- **table1 and table2:** The names of the two tables you want to join.
- **LEFT JOIN/ LEFT OUTER JOIN:** Specifies that you want to perform a LEFT OUTER JOIN operation.
- **ON:** Specifies the join condition defining the two tables' relation. It usually involves specifying the columns from each table that should be used for the join.

Example to Understand Left Outer Join in SQL Server.

Let us understand how to implement Left Outer Join in SQL Server with Examples. We will use the Company and Candidate tables to understand Left Outer Join in SQL Server.

Company		Candidate		
CompanyId	CompanyName	Candidat	FullName	Companyld
1	DELL	1	Ron	1
2	HP	2	Pete	2
3	IBM	3	Steve	3
4	Microsoft	4	Steve	NULL
		5	Ravi	1
		6	Raj	3
		7	Kiran	NULL

Please use the SQL script below to create the Company and Candidate tables and populate them with test data. Please note that the CompanyId column in the Candidate Table is a foreign key referencing the CompanyId column of the Company Table.

```
CREATE TABLE Company
(
    CompanyId TinyInt Identity Primary Key,
    CompanyName Nvarchar(50) NULL
)
```

```

GO
INSERT Company VALUES('DELL')
INSERT Company VALUES('HP')
INSERT Company VALUES('IBM')
INSERT Company VALUES('Microsoft')
GO
CREATE TABLE Candidate
(
    CandidateId tinyint identity primary key,
    FullName nvarchar(50) NULL,
    CompanyId tinyint REFERENCES Company(CompanyId)
)
GO
INSERT Candidate VALUES('Ron',1)
INSERT Candidate VALUES('Pete',2)
INSERT Candidate VALUES('Steve',3)
INSERT Candidate VALUES('Steve',NULL)
INSERT Candidate VALUES('Ravi',1)
INSERT Candidate VALUES('Raj',3)
INSERT Candidate VALUES('Kiran',NULL)
GO

```

Our requirement is to write a query to retrieve the CandidateId, FullName, CompanyId, and CompanyName from the Company and Candidate tables. The output of the query should be as shown below.

CandidateId	FullName	CompanyId	CompanyId	CompanyName
1	Ron	1	1	DELL
2	Pete	2	2	HP
3	Steve	3	3	IBM
4	Steve	NULL	NULL	NULL
5	Ravi	1	1	DELL
6	Raj	3	3	IBM
7	Kiran	NULL	NULL	NULL

Following is the SQL Query, an example of Left Outer Join that joins the Company and Candidate tables and will give the results shown in the above image.

```

SELECT Cand.CandidateId,
        Cand.FullName,
        Cand.CompanyId,
        Comp.CompanyId,
        Comp.CompanyName
FROM Candidate Cand
LEFT OUTER JOIN Company Comp
ON Cand.CompanyId = Comp.CompanyId;

```

Looking at the output, we got all 7 rows (i.e., all the rows from the Candidate Table), including the row with a null value for the CompanyId column. This proves that the SQL Server Left Outer Join will retrieve all the

rows from the Left-hand side Table, including the rows that have a null foreign key value.

Instead of using the **Left Outer Join** keyword, we can use the **Left Join** keyword. This will also work as expected as Left Outer Join. That means the Left Outer Join or Left Join means the same. The following SQL Query uses the Left Join Keyword to join the Candidate and Company tables.

```
SELECT Cand.CandidateId,
       Cand.FullName,
       Cand.CompanyId,
       Comp.CompanyId,
       Comp.CompanyName
FROM Candidate Cand
LEFT JOIN Company Comp
ON Cand.CompanyId = Comp.CompanyId;
```

Now, let us proceed and understand some tricky questions that may be asked in SQL Server interviews.

How Can We Retrieve Only Non-Matching Rows From The Left Table in SQL Server?

Here, what we want is, we want to retrieve only the non-matching records from the left table. For example, we want the following result sets when we join the Candidate and Company tables. If you notice, here we want to retrieve those rows from the Candidate table whose does not have any reference in the Company table.

CandidateId	FullName	CompanyId	CompanyId	CompanyName
4	Steve	NULL	NULL	NULL
7	Kiran	NULL	NULL	NULL

As we already discussed, the Left Outer Join fetches all the matching records from both tables and non-matching records from the left-hand side table. In this case, the non-matching records will take a NULL value. So, to get the above output, we need to perform Left Outer Join, and then in the where condition, we need to filter out the records with NULL values. The following Left Outer Join Query with the where clause does the same.

```
SELECT Cand.CandidateId,
       Cand.FullName,
       Cand.CompanyId,
       Comp.CompanyId,
       Comp.CompanyName
FROM Candidate Cand
LEFT JOIN Company Comp
ON Cand.CompanyId = Comp.CompanyId
WHERE Comp.CompanyId IS NULL;
```

Joining three Tables in SQL Server using Left Join:

Now, we will see how to join three tables in SQL Server. Already, we have the Company and Candidate tables. Let us introduce the third table, i.e., the Address table. So, we will use the Address, Company, and Candidate tables to perform joining operations between three tables in SQL Server.

AddressId	CandidateId	Country	State	City
1	1	India	Odisha	BBSR
2	2	India	Maharashtra	Mumbai
3	3	India	Maharashtra	Pune
4	4	India	Odisha	Cuttack

Please create and populate the Address table with the required sample data using the following SQL Script.

```

CREATE TABLE Address
(
    AddressId INT IDENTITY PRIMARY KEY,
    CandidateId tinyint REFERENCES Company(CompanyId),
    Country VARCHAR(50),
    State VARCHAR(50),
    City VARCHAR(50),
)
GO
INSERT INTO Address Values (1, 'India', 'Odisha', 'BBSR');
INSERT INTO Address Values (2, 'India', 'Maharashtra', 'Mumbai');
INSERT INTO Address Values (3, 'India', 'Maharashtra', 'Pune');
INSERT INTO Address Values (4, 'India', 'Odisha', 'Cuttack');
GO

```

The following is the syntax to join three tables in SQL Server.

```

SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name
LEFT JOIN table3
ON table1.column_name = table3.column_name;

```

Example: Left Joining Candidate, Company, and Address tables

The following Left Join SQL Query Joining Candidate, Company, and Address tables.

```

SELECT Cand.CandidateId,
       Cand.FullName,
       Cand.CompanyId,
       Comp.CompanyName,
       Addr.Country,
       Addr.State,
       Addr.City
FROM Candidate Cand

```

```
LEFT JOIN Company Comp
ON Cand.CompanyId = Comp.CompanyId
LEFT JOIN Address Addr
ON Addr.CandidateId = Cand.CandidateId;
```

You will get the following output when you execute the above SQL Query.

CandidateId	FullName	CompanyId	CompanyName	Country	State	City
1	Ron	1	DELL	India	Odisha	BBSR
2	Pete	2	HP	India	Maharashtra	Mumbai
3	Steve	3	IBM	India	Maharashtra	Pune
4	Steve	NULL	NULL	India	Odisha	Cuttack
5	Ravi	1	DELL	NULL	NULL	NULL
6	Raj	3	IBM	NULL	NULL	NULL
7	Kiran	NULL	NULL	NULL	NULL	NULL

Note: You need to use LEFT OUTER JOIN when you want to retrieve all the matching records from the tables involved in the JOIN and non-matching records from the left-hand side table.

When to use Left JOIN or Left Outer Join in SQL Server?

We need to use a LEFT JOIN (or LEFT OUTER JOIN, which is synonymous) in SQL Server or any other SQL-based database system when you want to retrieve data from two or more tables and include all the rows from the left (or first) table, regardless of whether there are matching rows in the right (or second) table. There are several scenarios where a LEFT JOIN is particularly useful:

- **Retrieving All Records from One Table Regardless of Matches:** Use LEFT JOIN when you want to retrieve all rows from the left table (first table in the JOIN clause), whether they have matching rows in the right table or not.
- **Analyzing Data Gaps:** It's useful for identifying which rows in the left table do not have corresponding entries in the right table, which can be critical for data analysis, auditing, or data integrity checks.
- **Combining Data from Multiple Sources:** When combining data from related tables, but you want to ensure that all records from a primary (left) table are included even if there are no corresponding entries in the secondary (right) table.
- **Creating Reports:** For reporting purposes where you need to show all items (e.g., all products) and their associated data (e.g., sales data), even if some items don't have any associated data (e.g., no sales).
- **Handling Optional Relationships:** Useful in scenarios where you have a relationship between two tables, but the relationship is optional, meaning some rows in the primary table may not have corresponding rows in the secondary table.

In the next article, I will discuss [Right Outer Join in SQL Server](#) with one real-time example. Here, in this article, I try to explain the **SQL Server Left Outer Join** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this article.